

1. Hausaufgabe

Aufgabe 1: Token Ring

Klonen Sie das Repository „TokenRingUDP-2025S“. Importieren Sie anschließend das enthaltene Java-Projekt in eine IDE Ihrer Wahl. Versuchen Sie den Source-Code zu verstehen und nutzen Sie bei Fragen und Problemen die Chat-Komponente des „Github Copilot“ oder eine andere generative KI. Versuchen Sie das Programm zu übersetzen und zu starten. Bei Problemen können Sie für erste Experimente auch direkt das fertige Fatjar im Wurzelverzeichnis des Projekts ausführen. Die Verwendung wird in der Datei *README.md* beschrieben.

Die Klasse **Token** stellt die Klassen zur Ringstruktur zur Verfügung, z.B. um neue Teilnehmer dem Ring hinzuzufügen, die Ring-Größe zu bestimmen oder die send/receive Klasse für Datenpakete.

Die Klasse **TokenRing** dient zur Erstellung eines Rings mit den definierten Methoden aus der Klasse **Token**. Der UDP-Ring wird mit einer while-Schleife erstellt und gibt regelmäßig die Größe des Rings und dessen Kandidaten aus. Außerdem enthält sie die main-Methode, mit der das Programm aufgerufen wird. Ohne Argument ist man der Ring Leader, mit IP-Adresse und Port kann man als Member dem UDP-Ring joinen.

Aufgabe 2: Teamwork

Wagen Sie sich zusammen mit anderen Vorlesungsteilnehmern an einen möglichst großen Ring. Die Protokollausgaben der Programme dokumentieren die erreichte Größe und die IP-Adressen der beteiligten Rechner.

Im 1. Versuch habe ich mehrere Members auf demselben PC dem Ring hinzugefügt, was man auch an der steigenden Anzahl der Members im UDP-Ring sieht, welche auf 5 anstieg:

```
Eingabeaufforderung
C:\Users\Andi\Desktop\TokenRingUDP-2025S>java -jar TokenRingUDP.jar
UDP endpoint is (192.168.178.174, 52061)
Received {"sequence":0,"ring":[{"ip":"192.168.178.174","port":60691}]} from 192.168.178.174:60691
Token: seq=0, #members=1 (192.168.178.174, 60691)
Sending {"sequence":1,"ring":[{"ip":"192.168.178.174","port":60691},{"ip":"192.168.178.174","port":52061}]} to 192.168.178.174:52061
Received {"sequence":1,"ring":[{"ip":"192.168.178.174","port":60691},{"ip":"192.168.178.174","port":52061}]} from 192.168.178.174:52061
Token: seq=1, #members=2 (192.168.178.174, 60691) (192.168.178.174, 52061)
Sending {"sequence":2,"ring":[{"ip":"192.168.178.174","port":52061},{"ip":"192.168.178.174","port":60691}]} to 192.168.178.174:60691
Received {"sequence":0,"ring":[{"ip":"192.168.178.174","port":60692}]} from 192.168.178.174:60692
Token: seq=0, #members=1 (192.168.178.174, 60692)
Received {"sequence":0,"ring":[{"ip":"192.168.178.174","port":60693}]} from 192.168.178.174:60693
Token: seq=0, #members=1 (192.168.178.174, 60693)
Received {"sequence":0,"ring":[{"ip":"192.168.178.174","port":60694}]} from 192.168.178.174:60694
Token: seq=0, #members=1 (192.168.178.174, 60694)
Received {"sequence":3,"ring":[{"ip":"192.168.178.174","port":60691},{"ip":"192.168.178.174","port":52061}]} from 192.168.178.174:60691
Token: seq=3, #members=2 (192.168.178.174, 60691) (192.168.178.174, 52061)
Sending {"sequence":4,"ring":[{"ip":"192.168.178.174","port":52061},{"ip":"192.168.178.174","port":60692},{"ip":"192.168.178.174","port":60693},{"ip":"192.168.178.174","port":60694}]} to 192.168.178.174:60691
Received {"sequence":5,"ring":[{"ip":"192.168.178.174","port":60692},{"ip":"192.168.178.174","port":60693},{"ip":"192.168.178.174","port":60694}]} from 192.168.178.174:60691
Token: seq=5, #members=5 (192.168.178.174, 60692) (192.168.178.174, 60693) (192.168.178.174, 60694) (192.168.178.174, 60691) (192.168.178.174, 52061)
^C
C:\Users\Andi\Desktop\TokenRingUDP-2025S>
```

Im 2. Versuch habe ich 2 Rechner (Kommilitone) untereinander kommunizieren lassen mit einigen Instanzen im UDP-Ring:

```
Eingabeaufforderung
C:\Users\Andi\Desktop\TokenRingUDP-2025S>java -jar TokenRingUDP.jar
UDP endpoint is (192.168.178.174, 59858)
Received {"sequence":0,"ring":[{"ip":"192.168.178.98","port":60744}]} from 192.168.178.98:60744
Token: seq=0, #members=1 (192.168.178.98, 60744)
Sending {"sequence":1,"ring":[{"ip":"192.168.178.98","port":60744},{"ip":"192.168.178.174","port":59858}]} to 192.168.178.174:59858
Received {"sequence":0,"ring":[{"ip":"192.168.178.98","port":60745}]} from 192.168.178.98:60745
Token: seq=0, #members=1 (192.168.178.98, 60745)
Received {"sequence":1,"ring":[{"ip":"192.168.178.98","port":60744},{"ip":"192.168.178.174","port":59858}]} from 192.168.178.174:59858
Token: seq=1, #members=2 (192.168.178.98, 60744) (192.168.178.174, 59858)
Sending {"sequence":2,"ring":[{"ip":"192.168.178.174","port":59858},{"ip":"192.168.178.98","port":60745}]} to 192.168.178.98:60744
Received {"sequence":0,"ring":[{"ip":"192.168.178.174","port":59859}]} from 192.168.178.174:59859
Token: seq=0, #members=1 (192.168.178.174, 59859)
Received {"sequence":3,"ring":[{"ip":"192.168.178.98","port":60745},{"ip":"192.168.178.98","port":60744}]} from 192.168.178.98:60744
Token: seq=3, #members=3 (192.168.178.98, 60745) (192.168.178.98, 60744) (192.168.178.174, 59858)
Sending {"sequence":4,"ring":[{"ip":"192.168.178.98","port":60744},{"ip":"192.168.178.174","port":59858}]} to 192.168.178.98:60745
Received {"sequence":0,"ring":[{"ip":"192.168.178.174","port":59860}]} from 192.168.178.174:59860
Token: seq=0, #members=1 (192.168.178.174, 59860)
Received {"sequence":6,"ring":[{"ip":"192.168.178.174","port":59859},{"ip":"192.168.178.98","port":60745}]} from 192.168.178.98:60745
Token: seq=6, #members=4 (192.168.178.174, 59859) (192.168.178.98, 60745) (192.168.178.98, 60744) (192.168.178.174, 59858)
Sending {"sequence":7,"ring":[{"ip":"192.168.178.98","port":60745},{"ip":"192.168.178.98","port":60744}]} to 192.168.178.174:59859
```

Aufgabe 3: WireShark

Installieren Sie die aktuelle Version des „Paket Sniffers“ WireShark. Fangen Sie mit der Hilfe dieses Programms einzelne Nachrichten eines Token-Rings aus Aufgabe 1 (lokal nur auf einem Rechner) sowie Aufgabe 2 (verschiedene Teilnehmer) ein. Machen Sie sich mit der Filter-Syntax von Wireshark vertraut, um nur die Token-Ring-Nachrichten darzustellen (Display Filter) oder überhaupt nur diese einzufangen (Capture Filter).

Im Folgenden Bild sind die abgefangenen lokalen Paketen aus dem 1. Teil der Aufgabe mit aktiviertem Anzeigefilter zu sehen. Wichtig hierbei ist es beim Adapter „loopback traffic capture“ (127.0.0.1) auszuwählen:

The screenshot shows the Wireshark interface with the title bar "*Adapter for loopback traffic capture". The menu bar includes Datei, Bearbeiten, Ansicht, Navigation, Aufzeichnen, Analyse, Statistiken, Telephonie, Wireless, Tools, and Hilfe. The toolbar contains various icons for file operations, navigation, and analysis. The packet list pane shows a list of 20 captured packets, all of which are UDP packets from 192.168.178.98 to 192.168.178.98 on port 50294. The packet details pane shows the selected packet (No. 77) with the following information:

- > Frame 77: 130 bytes on wire (1040 bits), 130 bytes captured (1040 bits) on interface \Device\NPF_{...}_Loopback, id 0
- > Null/Loopback
- > Internet Protocol Version 4, Src: 192.168.178.98, Dst: 192.168.178.98
- > User Datagram Protocol, Src Port: 50294, Dst Port: 50293
- > Data (98 bytes)

Beim 2. Teil der Aufgabe habe ich direkt zu Beginn den Capture Filter auf UDP eingestellt

Willkommen zu Wireshark

Aufzeichnen

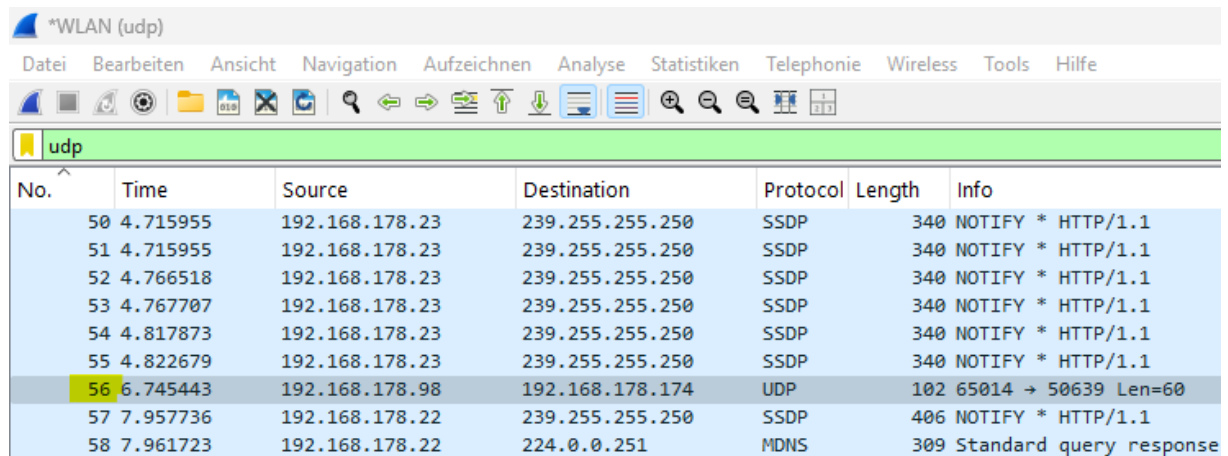
...mit diesem Filter: **udp**

- Ethernet
- Adapter for loopback traffic capture
- LAN-Verbindung* 8
- LAN-Verbindung* 7
- LAN-Verbindung* 6
- Event Tracing for Windows (ETW) reader

um dann die Kommunikation zwischen den Rechnern gefiltert aufzuzeichnen.

Zu Beginn gab es wie zu erwarten Schwierigkeiten bei der Kommunikation zwischen 2 Rechnern.

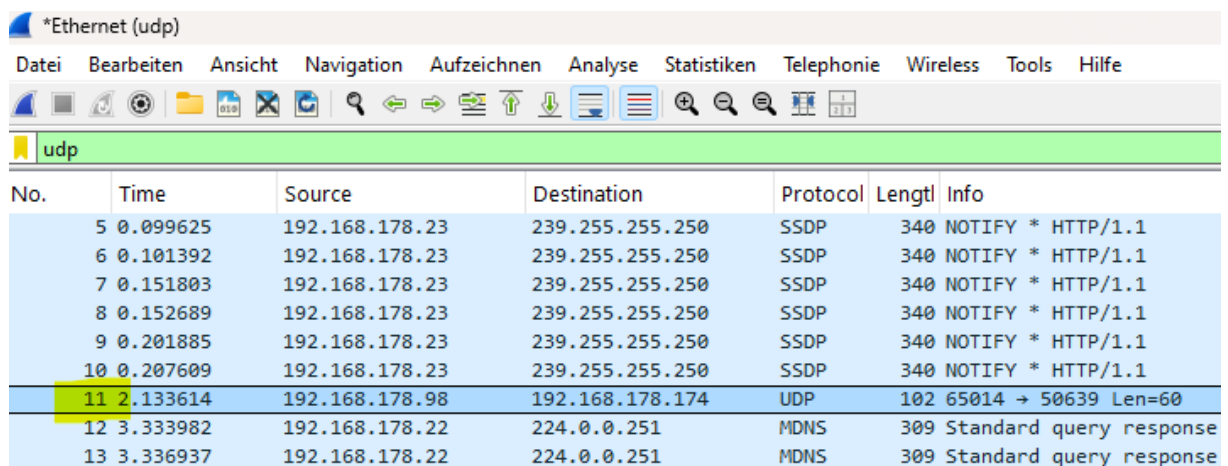
Wie in den nachfolgenden Bildern zu sehen ist, ging das Paket vom Sender aus ...



*WLAN (udp)

No.	Time	Source	Destination	Protocol	Length	Info
50	4.715955	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
51	4.715955	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
52	4.766518	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
53	4.767707	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
54	4.817873	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
55	4.822679	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
56	6.745443	192.168.178.98	192.168.178.174	UDP	102	65014 → 50639 Len=60
57	7.957736	192.168.178.22	239.255.255.250	SSDP	406	NOTIFY * HTTP/1.1
58	7.961723	192.168.178.22	224.0.0.251	MDNS	309	Standard query response

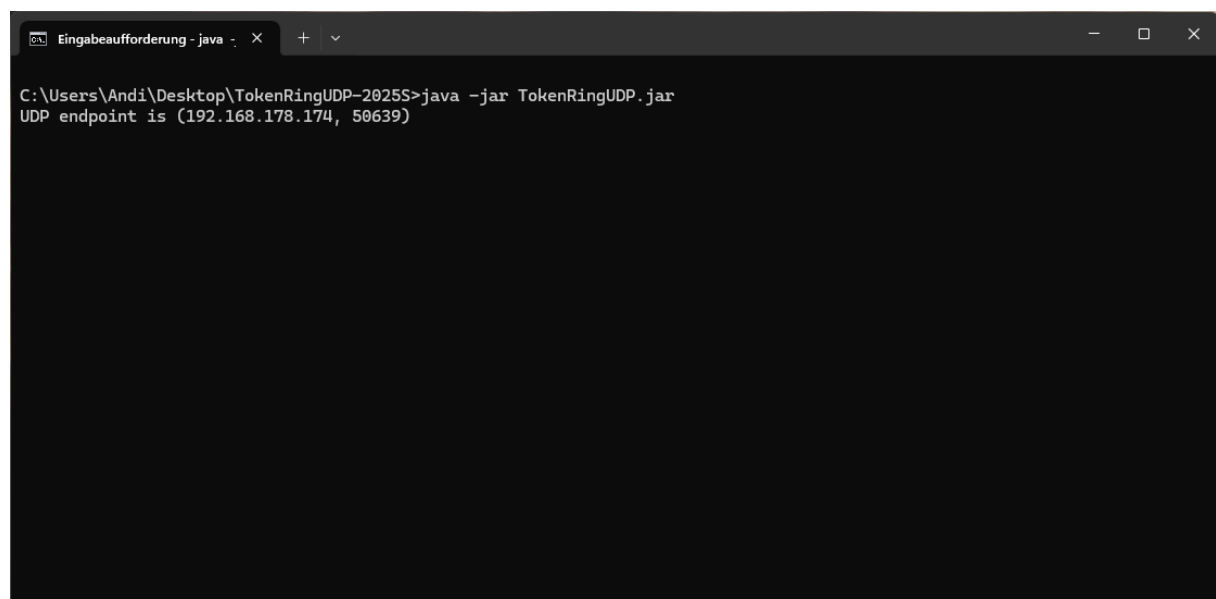
... und beim Empfänger ein ...



*Ethernet (udp)

No.	Time	Source	Destination	Protocol	Length	Info
5	0.099625	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
6	0.101392	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
7	0.151803	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
8	0.152689	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
9	0.201885	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
10	0.207609	192.168.178.23	239.255.255.250	SSDP	340	NOTIFY * HTTP/1.1
11	2.133614	192.168.178.98	192.168.178.174	UDP	102	65014 → 50639 Len=60
12	3.333982	192.168.178.22	224.0.0.251	MDNS	309	Standard query response
13	3.336937	192.168.178.22	224.0.0.251	MDNS	309	Standard query response

... aber dennoch wurde kein Member dem UDP-Ring hinzugefügt.

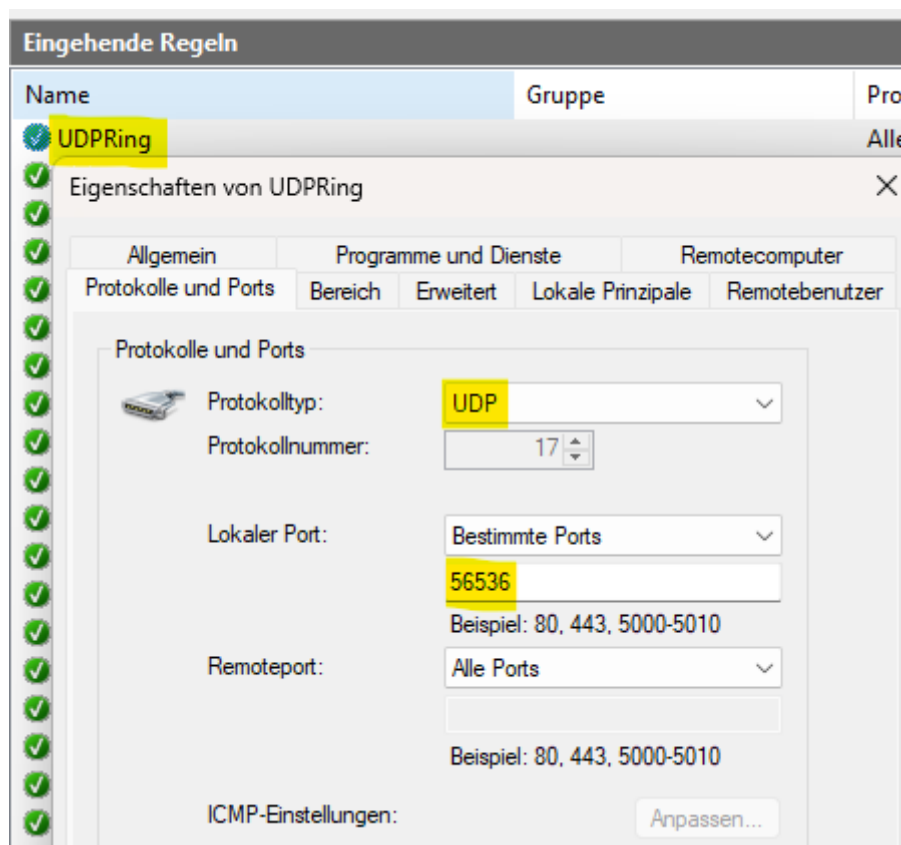


```
Eingabeaufforderung - java - x + v
C:\Users\Andi\Desktop\TokenRingUDP-2025S>java -jar TokenRingUDP.jar
UDP endpoint is (192.168.178.174, 50639)
```

Des Weiteren wartete der Sender auf eine Antwort vom Empfänger:

```
Eingabeaufforderung - java - x
C:\Users\Andi\Desktop>java -jar TokenRingUDP.jar 192.168.178.174 50639
UDP endpoint is (192.168.178.98, 55573)
Sending {"sequence":0,"ring":[{"ip":"192.168.178.98","port":55573}]} to 192.168.178.174:50639
```

Der Workaround darin bestand im Freigeben des spezifischen UDP-Ports in der **Windows Firewall des Empfängers**:



Dadurch wurde die Ein – und Ausgehende Kommunikation für den speziellen UDP-Port erlaubt und das hatte tatsächlich schon ausgereicht.

Eine Änderung der **Windows-Firewall vom Sender** war deshalb nicht erforderlich.

Somit kamen die Pakete beim Empfänger (192.168.178.174) an ...

```
Administrator: Eingabeaufforderung
C:\Users\Andi\Desktop>TokenRingUDP-20255>java -jar TokenRingUDP.jar
UDP endpoint is (192.168.178.174, 56536)
Received {"sequence":0,"ring":[{"ip":"192.168.178.98","port":55574}]} from 192.168.178.98:55574
Token: seq=0, #members=1 (192.168.178.98, 55574)
Sending {"sequence":1,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} to 192.168.178.174:56536
Received {"sequence":1,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} from 192.168.178.174:56536
Token: seq=1, #members=2 (192.168.178.98, 55574) (192.168.178.174, 56536)
Sending {"sequence":2,"ring":[{"ip":"192.168.178.174","port":56536}, {"ip":"192.168.178.98","port":55574}]} to 192.168.178.98:55574
Received {"sequence":3,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} from 192.168.178.98:55574
Token: seq=3, #members=2 (192.168.178.98, 55574) (192.168.178.174, 56536)
Sending {"sequence":4,"ring":[{"ip":"192.168.178.174","port":56536}, {"ip":"192.168.178.98","port":55574}]} to 192.168.178.98:55574
Received {"sequence":5,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} from 192.168.178.98:55574
Token: seq=5, #members=2 (192.168.178.98, 55574) (192.168.178.174, 56536)
Sending {"sequence":6,"ring":[{"ip":"192.168.178.174","port":56536}, {"ip":"192.168.178.98","port":55574}]} to 192.168.178.98:55574
Received {"sequence":7,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} from 192.168.178.98:55574
Token: seq=7, #members=2 (192.168.178.98, 55574) (192.168.178.174, 56536)
```

... und der Sender (192.168.178.98) erhielt eine Antwort, ...

```
Eingabeaufforderung - java - X
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\Andi>cd Desktop

C:\Users\Andi\Desktop>java -jar TokenRingUDP.jar 192.168.178.174 56536
UDP endpoint is (192.168.178.98, 55574)
Sending {"sequence":0,"ring":[{"ip":"192.168.178.98","port":55574}]} to 192.168.178.174:56536
Received {"sequence":2,"ring":[{"ip":"192.168.178.174","port":56536}, {"ip":"192.168.178.98","port":55574}]} from 192.168.178.174:56536
Token: seq=2, #members=2 (192.168.178.174, 56536) (192.168.178.98, 55574)
Sending {"sequence":3,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} to 192.168.178.174:56536
Received {"sequence":4,"ring":[{"ip":"192.168.178.174","port":56536}, {"ip":"192.168.178.98","port":55574}]} from 192.168.178.174:56536
Token: seq=4, #members=2 (192.168.178.174, 56536) (192.168.178.98, 55574)
Sending {"sequence":5,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} to 192.168.178.174:56536
Received {"sequence":6,"ring":[{"ip":"192.168.178.174","port":56536}, {"ip":"192.168.178.98","port":55574}]} from 192.168.178.174:56536
Token: seq=6, #members=2 (192.168.178.174, 56536) (192.168.178.98, 55574)
Sending {"sequence":7,"ring":[{"ip":"192.168.178.98","port":55574}, {"ip":"192.168.178.174","port":56536}]} to 192.168.178.174:56536
```

... was im Wireshark Protokoll folgendermaßen aussah:

*Ethernet (udp)						
Datei Bearbeiten Ansicht Navigation Aufzeichnen Analyse Statistiken Telefonie Wireless Tools Hilfe						
ip.addr == 192.168.178.98						
No.	Time	Source	Destination	Protocol	Length	Info
5	1.345988	192.168.178.98	192.168.178.174	UDP	102	58079 → 59792 Len=60
18	3.488499	192.168.178.174	192.168.178.98	UDP	140	59792 → 58079 Len=98
35	4.540215	192.168.178.98	192.168.178.174	UDP	140	58079 → 59792 Len=98
54	5.546754	192.168.178.174	192.168.178.98	UDP	140	59792 → 58079 Len=98
59	6.557138	192.168.178.98	192.168.178.174	UDP	140	58079 → 59792 Len=98
60	7.567076	192.168.178.174	192.168.178.98	UDP	140	59792 → 58079 Len=98
62	8.576197	192.168.178.98	192.168.178.174	UDP	140	58079 → 59792 Len=98
69	9.586594	192.168.178.174	192.168.178.98	UDP	140	59792 → 58079 Len=98
81	10.597242	192.168.178.98	192.168.178.174	UDP	140	58079 → 59792 Len=98
88	11.613570	192.168.178.174	192.168.178.98	UDP	141	59792 → 58079 Len=99
97	12.623201	192.168.178.98	192.168.178.174	UDP	141	58079 → 59792 Len=99
98	13.640874	192.168.178.174	192.168.178.98	UDP	141	59792 → 58079 Len=99
108	14.649615	192.168.178.98	192.168.178.174	UDP	141	58079 → 59792 Len=99